

Python Foundations for Agentic AI

Day 1 Course Notes — Python for Learners

with Java / C++ Background

Instructor

Dr. Amar Panchal

Career Credentials

Contact: 921-601163

CAREER CREDENTIALS | *Python Foundations for Agentic AI*

Course Contents

- Module 1 — Python Fundamentals (Fast Track)
- Module 2 — Decision Making & Loops
- Module 3 — Strings (Very Important)
- Module 4 — Collections (Most Important)
- Module 5 — Functions
- Module 6 — Comprehensions
- Module 7 — Modules & Packages
- Module 8 — Exception Handling
- Module 9 — File Handling
- Module 10 — Object-Oriented Programming
- Module 11 — Python Features Used in AI
- Module 12 — Working with Data
- Module 13 — Google Colab
- Module 14 — Python Libraries Every AI Developer Uses
- Module 15 — Python Best Practices

Note: This Day 1 course covers Python fundamentals through best practices. The transition into agentic AI frameworks, tool calling, and multi-agent concepts is addressed in the following session and is not part of this Python notes set.

Module 1 — Python Fundamentals (Fast Track)

Since you already know Java and C++, this module moves quickly. The focus is on where Python differs from the languages you know — no semicolons, no braces, no explicit type declarations.

1.1 Why Python for AI?

Python is the default language for AI and agentic development because almost every major AI library (OpenAI SDK, LangChain, LangGraph, CrewAI, PyTorch, TensorFlow) ships a Python-first API.

Compared to Java/C++, Python trades raw execution speed for development speed: less boilerplate, dynamic typing, and a huge ecosystem of ready-made packages via pip.

In an agentic AI workflow, you will spend most of your time wiring together API calls, JSON data, and control logic — Python is optimized for exactly this kind of "glue code."

1.2 Python Syntax and Indentation

Unlike Java/C++, Python uses indentation (whitespace) instead of curly braces { } to define blocks of code. There are no semicolons to end statements.

A colon (:) introduces a new block (after if, for, while, def, class, etc.), and everything indented under it belongs to that block. Standard indentation is 4 spaces — never mix tabs and spaces.

```
# Java/C++ style (NOT Python)
# if (x > 0) { System.out.println("positive"); }
```

```
# Python style
x = 5
if x > 0:
    print("positive") # 4-space indent defines the block
```

1.3 Variables and Dynamic Typing

You do not declare a variable's type. Python figures out the type at runtime based on the value assigned — this is called dynamic typing.

A variable can even be reassigned to a value of a different type later in the program (not recommended, but legal).

```
age = 25 # int
age = "25 years" # now a string — perfectly legal in Python
```

```
name = "Amar"
pi = 3.14
is_active = True
```

1.4 Input and Output

print() writes to the console. input() reads a line of text from the user and always returns a string, so you must convert it if you need a number.

```
name = input("Enter your name: ")
```

Dr. Amar Panchal • Career Credentials • Contact: 921-601163 • Page 3 of 29
CAREER CREDENTIALS | *Python Foundations for Agentic AI*

```
age = int(input("Enter your age: ")) # convert string -> int
print("Hello", name, "you are", age, "years old")
```

1.5 Data Types

Python's core built-in types:

- int — whole numbers, e.g. 10, -3
- float — decimal numbers, e.g. 3.14, -0.5
- bool — True or False (capitalized, unlike Java's true/false)
- string (str) — text, written in single or double quotes: 'hi' or "hi"
- None — Python's equivalent of null; represents "no value"

```
type(10) # <class 'int'>
type(3.14) # <class 'float'>
type(True) # <class 'bool'>
type("hello") # <class 'str'>
type(None) # <class 'NoneType'>
```

1.6 Operators

Python supports the same families of operators as Java/C++, with a few naming

differences: • Arithmetic: + - * / // (floor division) % ** (power)

- Comparison: == != > < >= <=
- Logical: and, or, not (words, not &&, ||, !)
- Assignment: = += -= *= /=
- Membership: in, not in
- Identity: is, is not

```
print(7 // 2) # 3 -> floor division
print(7 % 2) # 1 -> remainder
print(2 ** 5) # 32 -> power
print(5 in [1, 5, 9]) # True
```

1.7 Type Conversion

Use built-in functions to convert between types explicitly. This is common when reading user input or working with API responses (which often arrive as strings or JSON).

```
int("42") # 42
float("3.14") # 3.14
str(100) # "100"
bool(0) # False (0, "", None, [] are all "falsy")
bool(1) # True
```

1.8 Comments

Single-line comments start with `#`. Multi-line comments/docstrings use triple quotes `""" """` and are also used for documentation.

```
# this is a single-line comment
"""
```

```
This is a multi-line comment, often
used as a docstring. """
```

Module 2 — Decision Making & Loops

Control flow in Python follows the same logic as Java/C++, but with cleaner syntax — no parentheses required around conditions, and blocks are defined by indentation.

2.1 if Statement

```
age = 20
if age >= 18:
    print("Adult")
```

2.2 if-else

```
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

2.3 elif

Python's elif replaces Java/C++'s "else if" chains.

```
marks = 75
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "F"
print(grade)
```

2.4 Nested if

```
num = 15
if num > 0:
    if num % 2 == 0:
        print("Positive and even")
    else:
        print("Positive and odd")
```

2.5 for Loop

Python's for loop iterates directly over a sequence (list, string, range, etc.) — there is no traditional C-style `for(int i=0; i<n; i++)`.

```
for fruit in ["apple", "banana", "cherry"]:
    print(fruit)
```

```
for ch in "AI":
    print(ch)
```

2.6 while Loop

```
count = 0
while count < 5:
    print(count)
    count += 1
```

2.7 range()

`range()` generates a sequence of numbers and is most commonly used with for loops. `range(stop)`, `range(start, stop)`, and `range(start, stop, step)` are the three forms.

```
for i in range(5): # 0,1,2,3,4
    print(i)
```

```
for i in range(2, 10, 2): # 2,4,6,8
    print(i)
```

2.8 break

```
for i in range(10):
    if i == 5:
        break # exits the loop entirely
    print(i)
```

2.9 continue

```
for i in range(5):
    if i == 2:
        continue # skips this iteration only
    print(i)
```

2.10 pass

pass is a no-op placeholder used when a statement is syntactically required but you don't want any code to run yet — very useful while stubbing out functions during agent development.

```
def call_llm_api():
    pass # TODO: implement later
```

Module 3 — Strings (Very Important)

Strings are the backbone of AI development — prompts, API responses, and JSON payloads are all handled as strings. This module deserves close attention.

3.1 String Creation

```
s1 = 'single quotes'
s2 = "double quotes"
s3 = """triple quoted
spans multiple lines"""
```

3.2 Indexing

Strings are zero-indexed, exactly like Java/C++ arrays. Negative indices count from the end.

```
s = "Python"
print(s[0]) # 'P'
print(s[-1]) # 'n' (last character)
```

3.3 Slicing

Slicing extracts a substring: `s[start:stop:step]`. `stop` is exclusive.

```
s = "Agentic AI"
print(s[0:7]) # 'Agentic'
print(s[:7]) # same as above
print(s[8:]) # 'AI'
print(s[::-1]) # reverses the string
```

3.4 String Immutability

Strings cannot be modified in place — every "change" actually creates a new string object in memory. This is different from Java's `StringBuilder` mindset but similar to Java's `String` class itself.

```
s = "hello"
# s[0] = 'H' # ERROR: strings don't support item assignment
s = "H" + s[1:] # 'Hello' -> creates a NEW string
```

3.5 Common String Methods

- `split(sep)` — breaks a string into a list on a separator
- `join(iterable)` — opposite of `split`; joins a list into one string
- `replace(old, new)` — replaces all occurrences
- `strip()` — removes leading/trailing whitespace
- `lower()` / `upper()` — case conversion
- `startswith()` / `endswith()` — boolean checks on prefix/suffix

```
s = " Hello, Agentic AI World "
print(s.strip())
print(s.strip().lower())
words = s.strip().split(", ") # ['Hello', 'Agentic AI World']
print("-".join(["a", "b", "c"])) # 'a-b-c'
```

Dr. Amar Panchal • Career Credentials • Contact: 921-601163 • Page 8 of 29
CAREER CREDENTIALS | *Python Foundations for Agentic AI*

```
print("data.csv".endswith(".csv")) # True
```

3.6 f-strings

f-strings (formatted string literals) are the modern, preferred way to build strings with embedded variables — you will use these constantly to build prompts for LLMs.

```
name = "Amar"
score = 92.567
print(f"Hello {name}, your score is {score:.2f}")
# Output: Hello Amar, your score is 92.57
```

3.7 String Formatting

Older formatting styles you will still see in real code:

```
"Hello {}".format("Amar") # str.format()
"Hello %s" % "Amar" # printf-style (legacy)
```

3.8 Regular Expressions (Basic)

The `re` module lets you pattern-match text — useful for cleaning LLM output or extracting structured pieces from unstructured text.

```
import re
text = "Contact: amar@example.com"
match = re.search(r"[\w.]+@[ \w.]+", text)
print(match.group()) # amar@example.com

re.findall(r"\d+", "order 12 has 5 items") # ['12', '5']
```

Module 4 — Collections (Most Important)

Collections are how you'll store and pass data everywhere in agentic AI code — conversation history, tool outputs, API payloads. Dictionaries especially deserve deep attention since JSON maps directly onto them.

4.1 Lists — Overview

A list is an ordered, mutable (changeable) collection — the Python equivalent of Java's `ArrayList` or C++'s `vector`, but without fixed types.

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0]) # indexing: 'apple'
print(fruits[1:3]) # slicing: ['banana', 'cherry']
```

4.2 List Methods: `append`, `extend`, `insert`, `pop`, `remove`, `sort`

- `append(x)` — adds a single item to the end
- `extend(iterable)` — adds all items from another list
- `insert(i, x)` — inserts `x` at index `i`

- `pop(i)` — removes and returns the item at index `i` (default: last)
- `remove(x)` — removes the first matching value `x`
- `sort()` — sorts the list in place; `sort(reverse=True)` for descending

```
nums = [3, 1, 4]
nums.append(1) # [3, 1, 4, 1]
nums.extend([5, 9]) # [3, 1, 4, 1, 5, 9]
nums.insert(0, 100) # [100, 3, 1, 4, 1, 5, 9]
nums.pop() # removes last item (9)
nums.remove(1) # removes FIRST '1' found
nums.sort() # sorts ascending in place
```

4.3 Tuples — Why Immutable

A tuple is like a list but cannot be changed after creation. Because they're immutable, tuples are safer for fixed data (e.g. coordinates, function returns) and can be used as dictionary keys — lists cannot.

```
point = (10, 20)
# point[0] = 99 # ERROR: tuples don't support assignment
```

4.4 Tuple Packing and Unpacking

```
point = 10, 20 # packing (parentheses optional)
x, y = point # unpacking
print(x, y) # 10 20
```

```
def get_min_max(nums):
    return min(nums), max(nums)
lo, hi = get_min_max([4, 2, 9, 1])
```

4.5 Sets — Unique Values

A set stores only unique, unordered values — useful for deduplicating data (e.g. unique tags returned by an agent).

```
tags = {"ai", "python", "ai", "agent"}
print(tags) # {'ai', 'python', 'agent'} -> duplicates removed
```

4.6 Set Operations: Union, Intersection, Difference

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a | b) # union: {1, 2, 3, 4}
print(a & b) # intersection: {2, 3}
print(a - b) # difference: {1}
```

4.7 Dictionaries — Key-Value Concept

A dictionary stores data as key-value pairs — this is the single most important collection for AI work because JSON (used by every LLM API) maps one-to-one onto Python dictionaries.

Keys must be unique and immutable (strings, numbers, or tuples); values can be anything, including other dictionaries or lists.

4.8 Creating and Accessing Dictionaries

```
agent_config = {
    "name": "ResearchAgent",
```

```
"model": "claude-sonnet",
"temperature": 0.7
}

print(agent_config["name"]) # 'ResearchAgent'
print(agent_config.get("model")) # 'claude-sonnet'
```

4.9 get(), update(), pop()

- `get(key, default)` — safe access; returns default instead of raising an error if the key is missing
- `update(other_dict)` — merges another dictionary in, overwriting shared keys
- `pop(key)` — removes a key and returns its value

```
temp = agent_config.get("top_p", 1.0) # safe default if key not found
agent_config.update({"top_p": 0.9})
agent_config.pop("temperature")
```

4.10 items(), keys(), values()

```
for key, value in agent_config.items():
    print(key, "->", value)
```

```
print(list(agent_config.keys()))
print(list(agent_config.values()))
```

4.11 Nested Dictionaries

Real-world API responses (and agent memory structures) are almost always nested dictionaries and lists — this mirrors JSON exactly.

```
response = {
    "id": "msg_01",
    "usage": {"input_tokens": 120, "output_tokens": 45}, "content":
    [{"type": "text", "text": "Hello!"}] }
print(response["usage"]["input_tokens"]) # 120
print(response["content"][0]["text"]) # 'Hello!'
```

4.12 Iterating Dictionaries

```
for key in agent_config: # iterates over keys by default
    print(key, agent_config[key])
```

Module 5 — Functions

5.1 Defining Functions

```
def greet():  
    print("Hello!")
```

```
greet()
```

5.2 Parameters

```
def greet(name):  
    print(f"Hello, {name}!")  
greet("Amar")
```

5.3 Return Values

```
def add(a, b):  
    return a + b  
result = add(3, 4) # 7
```

5.4 Default Arguments

```
def call_model(prompt, temperature=0.7):  
    print(prompt, temperature)  
call_model("Hi") # uses default temperature=0.7  
call_model("Hi", 0.2)
```

5.5 Keyword Arguments

You can pass arguments by name, in any order — this is used heavily when calling AI SDK functions with many optional parameters.

```
call_model(prompt="Hi", temperature=0.2)  
call_model(temperature=0.2, prompt="Hi") # same result
```

5.6 Variable Arguments (*args, **kwargs)

*args collects any number of extra positional arguments into a tuple. **kwargs collects any number of extra keyword arguments into a dictionary. Both are extremely common in AI SDKs and decorator/wrapper functions.

```
def log_call(*args, **kwargs):  
    print("positional:", args)  
    print("keyword:", kwargs)  
  
log_call(1, 2, model="sonnet", stream=True)  
# positional: (1, 2)  
# keyword: {'model': 'sonnet', 'stream': True}
```

5.7 Scope

Variables defined inside a function are local and not visible outside it. Variables defined outside are global; use the global keyword only if you must modify a global from inside a function (rare and generally avoided).

```
x = 10 # global  
def show():  
    x = 5 # local, separate variable  
    print(x)  
show() # 5  
print(x) # 10 (unchanged)
```

5.8 Lambda (Anonymous Functions)

```
square = lambda x: x * x  
print(square(5)) # 25
```

5.9 map()

```
nums = [1, 2, 3, 4]  
doubled = list(map(lambda x: x * 2, nums)) # [2, 4, 6, 8]
```

5.10 filter()

```
nums = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums)) # [2, 4, 6]
```

5.11 zip()

zip() pairs up items from multiple sequences — useful for combining a list of prompts with a list of expected outputs.

```
names = ["Alice", "Bob"]
scores = [90, 85]
for name, score in zip(names, scores):
    print(name, score)
```

5.12 enumerate()

enumerate() gives you both the index and the value while looping — cleaner than manually tracking a counter.

```
for i, fruit in enumerate(["apple", "banana"]):
    print(i, fruit)
# 0 apple
# 1 banana
```

Module 6 — Comprehensions

Comprehensions let you build lists, dictionaries, and sets in a single readable line instead of a multi-line loop — Python code in AI projects uses these constantly.

6.1 List Comprehension

```
squares = [x * x for x in range(6)]
# [0, 1, 4, 9, 16, 25]
```

```
evens = [x for x in range(20) if x % 2 == 0]
```

6.2 Dictionary Comprehension

```
words = ["ai", "agent", "llm"]
lengths = {w: len(w) for w in words}
# {'ai': 2, 'agent': 5, 'llm': 3}
```

6.3 Set Comprehension

```
nums = [1, 2, 2, 3, 3, 3]
unique_squares = {x * x for x in nums}
```

```
# {1, 4, 9}
```

Module 7 — Modules & Packages

7.1 import

```
import math  
print(math.sqrt(16)) # 4.0
```

7.2 from ... import

```
from math import sqrt, pi  
print(sqrt(16))  
print(pi)
```

7.3 Aliases

```
import numpy as np # common convention  
import pandas as pd
```

7.4 Creating Your Own Module

Any .py file is automatically a module. If you save helper functions in utils.py, you can import them from another file in the same folder with: import utils, or from utils import my_function.

7.5 if __name__ == "__main__"

This block only runs when the file is executed directly (not when imported by another file) — the Python equivalent of a main() entry point guard.

```
def main():
    print("Running as a script")

if __name__ == "__main__":
    main()
```

7.6 Standard Libraries

- math — mathematical functions (sqrt, floor, ceil, etc.)
- random — random number generation, useful for sampling/testing
- json — encode/decode JSON, essential for working with LLM APIs
- os — interact with the operating system (paths, environment variables)
- pathlib — modern, object-oriented file path handling
- datetime — dates, times, and timestamps
- collections — extra data structures (Counter, defaultdict, namedtuple)

Dr. Amar Panchal • Career Credentials • Contact: 921-601163 • Page 16 of 29
CAREER CREDENTIALS | *Python Foundations for Agentic AI*

Module 8 — Exception Handling

In agentic AI code, API calls fail, rate limits get hit, and JSON parsing breaks. Robust exception handling is essential, not optional.

8.1 try

Code that might fail goes inside a try block.

8.2 except

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

8.3 finally

finally always runs, whether or not an exception occurred — used for cleanup like closing files or connections.

```
try:
    response = call_api()
except Exception as e:
    print("Error:", e)
finally:
    print("Request attempt finished")
```

8.4 else

The else block runs only if no exception was raised.

```
try:
    value = int("42")
except ValueError:
    print("Invalid number")
else:
    print("Parsed successfully:", value)
```

8.5 raise

Use raise to manually trigger an exception, e.g. when validating input.

```
def set_temperature(t):
    if not (0 <= t <= 1):
        raise ValueError("temperature must be between 0 and 1")
```

8.6 Custom Exceptions (Brief)

```
class RateLimitError(Exception):
    pass
```

```
raise RateLimitError("Too many requests")
```

Module 9 — File Handling

9.1 Reading Files

```
file = open("notes.txt", "r")
content = file.read()
file.close()
```

9.2 Writing Files

```
file = open("output.txt", "w")
file.write("Hello from Python")
file.close()
```

9.3 with Statement

The with statement automatically closes the file for you, even if an error occurs — this is the preferred, safe way to work with files (equivalent to try-with-resources in Java).

```
with open("notes.txt", "r") as file:
    content = file.read()
# file is automatically closed here
```

9.4 CSV

```
import csv
with open("data.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)
```

9.5 JSON

This is one of the most-used patterns in AI development: reading/writing JSON files for configs, prompts, and cached API responses.

```
import json
```

```
with open("config.json", "r") as file:  
    config = json.load(file) # JSON file -> Python dict
```

```
with open("config.json", "w") as file:  
    json.dump(config, file, indent=2) # Python dict -> JSON file
```

Module 10 — Object-Oriented Programming

You already know OOP from Java/C++, so this module focuses only on Python's syntax and just enough depth to read and use AI libraries, which are built almost entirely out of classes.

10.1 Classes and 10.2 Objects

```
class Agent:  
    pass
```

```
my_agent = Agent() # object / instance of the class
```

10.3 Constructors (__init__)

`__init__` is Python's constructor — it runs automatically when an object is created.

```
class Agent:  
    def __init__(self, name, model):  
        self.name = name # instance variable  
        self.model = model
```

```
a1 = Agent("ResearchAgent", "claude-sonnet")
```

10.4 Instance Variables and 10.5 Methods

`self` refers to the current object — it must be the first parameter of every instance method (Python's equivalent of Java/C++'s implicit 'this').

```
class Agent:  
    def __init__(self, name):  
        self.name = name  
  
    def introduce(self):  
        print(f"I am {self.name}, an AI agent.")
```

```
a1 = Agent("ResearchAgent")  
a1.introduce()
```

10.6 Inheritance

```
class BaseAgent:
    def __init__(self, name):
        self.name = name
    def act(self):
        print(f'{self.name} is acting')
```

```
class ResearchAgent(BaseAgent): # inherits from BaseAgent
    def act(self): # 10.7 Polymorphism: overrides parent method
        print(f'{self.name} is researching')
```

```
agents = [BaseAgent("A"), ResearchAgent("B")]
for a in agents:
    a.act() # each object calls its own version of act()
```

Dr. Amar Panchal • Career Credentials • Contact: 921-601163 • Page 19 of 29
CAREER CREDENTIALS | *Python Foundations for Agentic AI*

10.8 Encapsulation

Python has no strict 'private' keyword. By convention, a single leading underscore (`_var`) signals 'internal use only', and a double leading underscore (`__var`) triggers name-mangling for stronger protection.

```
class Agent:
    def __init__(self, api_key):
        self.__api_key = api_key # name-mangled, harder to access externally

    def get_masked_key(self):
        return self.__api_key[:4] + "*****"
```

10.9 Special Methods (`__init__`, `__str__`)

Special ("dunder") methods let your objects work with built-in Python syntax. `__str__` controls what `print()` shows for your object — you will see many dunder methods used across AI libraries (e.g. `__call__`, `__repr__`).

```
class Agent:
    def __init__(self, name):
        self.name = name
    def __str__(self):
        return f"<Agent: {self.name}>"

print(Agent("Bot1")) # <Agent: Bot1>
```

Module 11 — Python Features Used in AI

11.1 Iterators

An iterator is an object you can step through one value at a time using `next()`. Any object with `__iter__` and `__next__` is an iterator. Lists, dicts, and strings are 'iterable' — you can get an iterator from them.

```
nums = [1, 2, 3]
it = iter(nums)
print(next(it)) # 1
print(next(it)) # 2
```

11.2 Generators and 11.3 yield

A generator is a function that produces values lazily (one at a time, on demand) using `yield` instead of `return`. This is extremely important for AI work because it's exactly how streaming LLM responses are handled — tokens arrive one chunk at a time instead of all at once.

```
def stream_tokens(tokens):
    for t in tokens:
        yield t # pauses here, resumes on next call
```

```
for token in stream_tokens(["Hello", " ", "World"]):
    print(token, end="")
```

11.4 Decorators (Basic)

A decorator wraps a function to add extra behavior without modifying the function itself — commonly used in AI frameworks to register tools/functions (e.g. `@tool`, `@app.route`).

```
def log_calls(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper
```

```
@log_calls
def greet(name):
    print(f"Hello {name}")
```

```
greet("Amar") # prints the log line, then the greeting
```

11.5 Type Hints

Type hints are optional annotations that document expected types — Python does not enforce them at runtime, but tools and AI SDKs rely on them heavily for validation and auto-generated documentation.

```
def add(a: int, b: int) -> int:
    return a + b
```

```
def get_config() -> dict:
    return {"model": "sonnet"}
```

11.6 Dataclasses

The `@dataclass` decorator auto-generates `__init__`, `__repr__`, and other boilerplate for classes that mainly hold data — very common for defining structured agent inputs/outputs.

```
from dataclasses import dataclass
```

```
@dataclass
class AgentResponse:
    text: str
    tokens_used: int
```

```
r = AgentResponse(text="Hello", tokens_used=12)
print(r) # AgentResponse(text='Hello', tokens_used=12)
```

11.7 Named Tuples

A named tuple is a lightweight, immutable data structure with named fields — like a tuple, but readable by name.

```
from collections import namedtuple
```

```
Point = namedtuple("Point", ["x", "y"])
p = Point(10, 20)
print(p.x, p.y) # 10 20
```

Module 12 — Working with Data

12.1 JSON

JSON (JavaScript Object Notation) is the standard data format for nearly all AI APIs. Structurally, a JSON object maps to a Python dict, and a JSON array maps to a Python list.

12.2 Dictionary <-> JSON Conversion

```
import json
```

```
data = {"name": "Agent1", "tools": ["search", "calculator"]}
```

```
json_string = json.dumps(data) # dict -> JSON string  
print(json_string)
```

```
parsed = json.loads(json_string) # JSON string -> dict  
print(parsed["tools"])
```

12.3 REST API Basics

Most AI services expose a REST API: you send an HTTP request (commonly POST, with a JSON body) to an endpoint URL, and receive a JSON response back.

Key concepts: endpoint (URL), method (GET/POST/PUT/DELETE), headers (e.g. Authorization, Content Type), and body (the JSON payload).

12.4 requests Library

requests is the standard Python library for making HTTP calls — this is how you'll call most AI APIs directly. `import requests`

```
response = requests.post(
    "https://api.example.com/v1/messages",
    headers={"Authorization": "Bearer YOUR_KEY"},
    json={"model": "claude-sonnet", "prompt": "Hello"}
)
print(response.status_code) # e.g. 200
data = response.json() # parse JSON response body
```

12.5 Parsing API Responses

```
# Typical nested response structure
data = response.json()
text = data["content"][0]["text"]
tokens = data["usage"]["output_tokens"]
print(text, tokens)
```

Dr. Amar Panchal • Career Credentials • Contact: 921-601163 • Page 23 of 29
CAREER CREDENTIALS | *Python Foundations for Agentic AI*

Module 13 — Google Colab

Google Colab is a free, browser-based Python notebook environment with GPU access — the standard environment for experimenting with AI code without installing anything locally.

13.1 Notebook Interface

A Colab notebook is made of cells — either code cells (Python) or text cells (Markdown) — arranged top to bottom.

13.2 Running Cells

Press Shift+Enter (or the ► button) to run the current cell. Cells can be run in any order, but running top to-bottom is standard practice to keep state predictable.

13.3 Markdown Cells

Text cells support Markdown formatting (headings, bold, lists, links) for writing notes and explanations alongside your code.

13.4 Installing Packages (!pip)

A leading ! runs a shell command inside the notebook. This is how you install any package not already available in Colab.

```
!pip install openai langchain
```

13.5 Uploading Files

```
from google.colab import files
```

```
uploaded = files.upload() # opens a file picker dialog
```

13.6 Downloading Files

```
from google.colab import files  
files.download("output.json")
```

13.7 Mounting Google Drive

```
from google.colab import drive  
drive.mount('/content/drive')  
# files now accessible under /content/drive/MyDrive/
```

13.8 Runtime Selection

Runtime > Change runtime type lets you choose the Python version and hardware accelerator (None / GPU / TPU) for the session.

13.9 GPU Usage (Brief)

Select GPU under runtime type for tasks like model fine-tuning or running local models — not usually required for calling hosted LLM APIs, which run on the provider's servers, not yours.

Module 14 — Python Libraries Every AI Developer Uses

A quick-reference map of the standard libraries you already met in this course, plus what each one is used for in day-to-day agentic AI work.

14.1 Standard Library Quick Reference

- os — read environment variables (e.g. API keys), work with file paths
- pathlib — modern, cleaner alternative to os.path for handling file paths
- json — encode/decode JSON for API requests and responses
- requests — make HTTP calls to REST APIs (LLM providers, tools, webhooks)
- datetime — timestamps for logging, scheduling agent tasks
- re — regular expressions, for cleaning/extracting text from model output
- math — numeric utilities (rarely central, but used in scoring/metrics)
- random — sampling, shuffling, generating test data
- collections — Counter, defaultdict, namedtuple for cleaner data handling
- typing — type hints (List, Dict, Optional, etc.) for clear function signatures
- dataclasses — structured, boilerplate-free data containers for agent I/O

14.2 Typical Import Block for an Agentic AI Script

```
import os
import json
import requests
from pathlib import Path
from datetime import datetime
from dataclasses import dataclass
from typing import List, Optional
```

Module 15 — Python Best Practices

15.1 PEP 8

PEP 8 is Python's official style guide — following it keeps your code readable and consistent with the rest of the Python community (similar in spirit to standard Java/Google style guides).

- 4 spaces per indentation level (no tabs)
- Maximum line length around 79–99 characters
- One blank line between logically distinct blocks; two around top-level functions/classes

15.2 Naming Conventions

- variables and functions: snake_case (get_response, api_key)
- classes: PascalCase (ResearchAgent, ApiClient)
- constants: ALL_CAPS (MAX_TOKENS, DEFAULT_MODEL)
- private/internal: leading underscore (_helper_function)

15.3 Writing Reusable Functions

Keep functions small and single-purpose. If a function does more than one clear job (e.g. 'call API' AND 'parse response' AND 'log result'), split it into smaller functions that are easier to test and reuse across agents.

15.4 Docstrings

A docstring documents what a function does, its parameters, and its return value — placed as the first statement inside the function, using triple quotes.

```
def call_model(prompt: str, temperature: float = 0.7) -> str:
    """
    Sends a prompt to the LLM and returns the text response.
```

```
    Args:
    prompt: the input text to send to the model.
    temperature: creativity/randomness, 0 to 1.
```

```
    Returns:
    The model's text response as a string.
    """
    pass
```

15.5 Debugging

- Use print() statements strategically to inspect variable values at key points
- Read the full traceback bottom-up — the last line names the actual error
- Use try/except around API calls during development to see the real error message
- Colab and most IDEs support breakpoints and a step-through debugger for deeper issues

15.6 Virtual Environments (Concept Only)

A virtual environment is an isolated Python installation for a single project, so its package versions don't conflict with other projects on the same machine — conceptually similar to per-project dependency management in Maven/Gradle.

Created with python -m venv env and activated before installing project-specific packages. In Colab, each notebook session is already isolated, so this is rarely needed there.



Welcome to **Amar Panchal's World**



One Scan,
All Connections



Stay Updated,
Stay Inspired



Connect.
Learn. Grow.



Your Growth
Partner



Save. Follow.
Stay Ahead.



Let's Build
the Future
Together!

Let's Connect!

➤ Scan this QR to connect with
Dr. Amar Panchal ➤

Scan. Connect. Succeed.



**Every Step
tiny.**

h - we transform.
red learners,
sible citizens

skills,
better



Empowering Minds.
Enriching Futures.
Building Digital India.

**Be the Change.
Lead the Future.**



Your
Journey.

CAREER CREDENTIALS | Python
Foundations for Agentic AI

